Multimodal Table Scanning Application

Multimodal Table Scanning Application
- 1. Concept Introduction
  - 1.1 What is "Multimodal Table Scanning"?
  - 1.2 Implementation Principle Summary
- 2. Code Analysis
  - Key Code
    - 1. Tool Layer Entry (`largemodel/utils/tools_manager.py`)
    - 2. Model Interface Layer (`largemodel/utils/large_model_interface.py`)
  - Code Analysis
- 3. Practical Operations
  - 3.1 Configure Online LLM
  - 3.2 Start and Test the Feature

1. Concept Introduction

1.1 What is "Multimodal Table Scanning"?

Multimodal Table Scanning is a technology that utilizes image processing and artificial intelligence techniques to recognize and extract tabular information from images or PDF documents. It not only focuses on visual table structure recognition but also combines multimodal data such as text content and layout information to enhance the understanding of tables. **Large Language Models (LLMs)** provide powerful semantic analysis capabilities in understanding this extracted information. The two complement each other, together improving the intelligence level of document processing.

1.2 Implementation Principle Summary

1. **Table Detection and Content Recognition**
   - Computer vision technology is used to locate tables in documents, and OCR technology converts the text within the tables into an editable format.
   - Deep learning methods are used to parse the table structure (row/column division, merged cells, etc.) to generate structured data representations.
2. **Multimodal Fusion**
   - Visual (such as table layout), textual (OCR results), and potentially existing metadata (such as file type, source) are integrated to form a comprehensive data view.
   - Specially designed multimodal models (e.g., LayoutLM) are used to process these different types of data simultaneously to more accurately understand table content and its contextual relationships.

2. Code Analysis

Key Code

1. Tool Layer Entry (`largemodel/utils/tools_manager.py`)

The `scan_table` function in this file defines the execution flow of this tool, especially how it constructs a Prompt requiring Markdown format output.

```
# From largemodel/utils/tools_manager.py
class ToolsManager:
    # ...
    def scan_table(self, args):
        """
        Scan a table from an image and save the content as a Markdown file.

        :param args: Arguments containing the image path.
        :return: Dictionary with file path and content.
        """
        self.node.get_logger().info(f"Executing scan_table() tool with args:
{args}")
        try:
            image_path = args.get("image_path")
            # ... (Path checking and fallback)

            # Construct a prompt asking the large model to recognize the table
and return it in Markdown format.
            if self.node.language == 'zh':
                prompt = "请仔细分析这张图片，识别其中的表格，并将其内容以Markdown格式返
回。"
            else:
                prompt = "Please carefully analyze this image, identify the
table within it, and return its content in Markdown format."

            result = self.node.model_client.infer_with_image(image_path, prompt)

            # ... (Extract Markdown text from result)

            # Save the recognized content to a Markdown file.
            md_file_path = os.path.join(self.node.pkg_path, "resources_file",
"scanned_tables", f"table_{timestamp}.md")
            with open(md_file_path, 'w', encoding='utf-8') as f:
                f.write(table_content)

            return {
                "file_path": md_file_path,
                "table_content": table_content
            }
        # ... (Error handling)
```

2. Model Interface Layer (`largemodel/utils/large_model_interface.py`)

The `infer_with_image` function in this file is the unified entry point for all image-related tasks.

```
# From largemodel/utils/large_model_interface.py

class model_interface:
```

```
    # ...
    def infer_with_image(self, image_path, text=None, message=None):
        """Unified image inference interface."""
        # ... (Prepare message)
        try:
            # Based on the value of self.llm_platform, decide which specific
implementation to call
            if self.llm_platform == 'ollama':
                response_content = self.ollama_infer(self.messages,
image_path=image_path)
            elif self.llm_platform == 'tongyi':
                # ... Call Tongyi model logic
                pass
            # ... (Other platform logic)
        # ...
        return {'response': response_content, 'messages': self.messages.copy()}
```

Code Analysis

The table scanning feature is a typical application of converting unstructured image data into structured text data. Its core technology is still **using Prompt Engineering to guide model behavior**.

1. **Tool Layer (`tools_manager.py`)**:
    - The `scan_table` function is the business process controller of this feature. It receives an image containing a table as input.
    - The most critical operation of this function is **constructing a highly targeted Prompt**. This Prompt directly instructs the large model to perform two tasks: 1. Identify the table in the image. 2. Return the identified content in Markdown format. This mandatory requirement for output format is the key to achieving the unstructured to structured conversion.
    - After constructing the Prompt, it calls the `infer_with_image` method of the model interface layer, passing the image and this formatting instruction together.
    - After receiving the Markdown text returned from the model interface layer, it performs a file operation: writing this text content into a new `.md` file.
    - Finally, it returns structured data containing the new file path and table content.
2. **Model Interface Layer (`large_model_interface.py`)**:
    - The `infer_with_image` function continues as the unified "dispatch center". It receives the image and Prompt from `scan_table`, and based on the current system configuration (`self.llm_platform`), dispatches the task to the correct backend model implementation.
    - Regardless of the backend model, this layer's task is to handle communication details with the specific platform, ensuring the image and text data are sent correctly, and then returning the plain text returned by the model (in this case, Markdown-formatted text) to the tool layer.

To summarize, the general flow of table scanning is: `ToolsManager` receives the image and constructs an instruction "convert the table in this image to Markdown" -> `ToolsManager` calls the model interface -> `model_interface` packages the image and instruction together and sends them to the corresponding model platform based on configuration -> The model returns Markdown-formatted text -> `model_interface` returns the text to `ToolsManager` -> `ToolsManager` saves the text as a `.md` file and returns the result. This flow demonstrates how

the large model's format-following capability can be leveraged to use it as a powerful OCR (Optical Character Recognition) and data structuring tool.

3. Practical Operations

3.1 Configure Online LLM

1. **Update the API key in the configuration file, open the model interface configuration file** `large_model_interface.yaml`:

   ```
   vim ~/yahboomcar_ws/src/largemodel/config/large_model_interface.yaml
   ```

2. **Enter your API Key**:
   Find the corresponding section and paste your copied API Key here. Here's an example using Tongyi Qianwen configuration:

   ```
   # large_model_interface.yaml

   ## Tongyi Qianwen
   qianwen_api_key: "sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx" # Paste your Key here
   qianwen_model: "qwen-vl-max-latest" # Can choose model as needed, such as
   qwen-turbo, qwen-plus
   ```

3. **Open the main configuration file** `yahboom.yaml`:

   ```
   vim ~/yahboomcar_ws/src/largemodel/config/yahboom.yaml
   ```

4. **Select the online platform to use**:
   Modify the `llm_platform` parameter to the platform name you want to use:

   ```
   # yahboom.yaml

   model_service:
     ros__parameters:
       # ...
       llm_platform: 'tongyi'  #Optional platforms:'tongyi', 'spark',
   'qianfan', 'openrouter'
   ```

   After modifying the configuration file, you need to recompile in the workspace and source:

   ```
   cd ~/yahboomcar_ws
   colcon build && source install/setup.bash
   ```

3.2 Start and Test the Feature

1. **Prepare a table image file**:

   Place a table image file you want to test in the following path:
   `~/yahboomcar_ws/src/largemodel/resources_file/scan_table`

   Then rename the image to `test_table.jpg`

2. Start the `largemodel` main program:

   Open a terminal and run the following command:

```
ros2 launch largemodel largemodel_control.launch.py text_chat_mode:=true
```

3. **Send text instructions**:

   Due to the official DiGua (RDK) system, Chinese input method can only be used when the system language is set to Chinese. The factory image does not switch languages. You can use Chinese pinyin input via the SSH remote software terminal.

   Open another terminal and run the following command:

   ```
   ros2 run text_chat text_chat
   ```

   Then start entering text: "分析一下表格" (Analyze the table)

4. **Observe the results**:
   In the first terminal running the main program, you will see log output showing the system received the instruction, called the `scan_table` tool, prompted that `scan_table` execution is complete, and the scanned information was saved to a document.

   You can find this document in the `~/yahboomcar_ws/src/largemodel/resources_file/scan_table` path.